

Table of Contents

1. Project Overview	3
1.1 Key Features	3
2. Technology Stack	4
2.1 Core Technologies	4
2.2 Supporting Libraries	5
3. Database Schema	5
3.1 Entity Overview	5
3.2 Role-Based Access Control	6
4. Project Structure	6
4.1 Directory Layout	6
5. API Design	7
5.1 API Endpoints	8
5.2 Response Format	8
6. AI Assistant Integration	9
6.1 Chat Protocol	9
6.2 Future: Agent Mode	9
7. Time Tracking System	9
7.1 Clock In/Out Flow	10

7.2 Time Log Analytics	10
8. Notification System	10
9. Security Architecture	11
10. Deployment Guide	12
10.1 Environment Variables	12
10.2 Deployment Checklist	12
11. Coding Conventions	13
11.1 Key Rules	13
12. Using AGENT.md for Persistent Memory	13
12.1 What AGENT.md Contains	14
12.2 How to Use It	14
13. Credentials and Setup Requirements	15

1. Project Overview

TeamForge PM is a full-stack project management web application designed to help teams collaborate efficiently. It provides a comprehensive suite of tools for managing team members, tracking project progress, monitoring work hours through a built-in time tracking system, and leveraging artificial intelligence for project development assistance. The platform is built with a modern technology stack centered around Next.js, TypeScript, and Turso SQLite, ensuring high performance, reliability, and scalability for teams of all sizes.

The application follows a clear role-based access control model with three distinct roles: Superadmin, Admin, and Member. The Superadmin has unrestricted access to the entire system and can manage users, projects, and system configuration. Admins can create and manage projects, assign team members, and review time logs. Members can clock in and out of work sessions, view their assigned projects, and interact with the AI assistant. This hierarchy ensures proper separation of concerns while maintaining operational flexibility.

One of the standout features of TeamForge PM is its integration with GLM (the AI model) via the z-ai-web-dev-sdk. This integration allows team members to chat with an AI assistant that has full context about their projects, helping with code generation, documentation drafting, problem-solving, and more. In the future, the AI agent mode will enable GLM to directly access Git repositories and deploy changes through Vercel, creating an end-to-end automated development workflow right from the project management interface.

1.1 Key Features

Feature	Description
Authentication	Secure email/password login with JWT sessions via NextAuth.js v5
Role-Based Access	Three-tier permission system (Superadmin, Admin, Member)
Dashboard	Real-time overview of projects, team activity, and key metrics
User Management	Add, edit, and deactivate team members with role assignment
Project Management	Create, organize, and track projects with status management
Team Assignment	Assign leads and members to projects with project-specific roles
Time Tracking	Clock in/out with duration calculation and project selection
AI Assistant	GLM-powered chat interface for project help and code generation
Notifications	Email alerts via Resend for invitations, assignments, and summaries

Audit Logging	Complete activity trail for login, logout, and system actions
---------------	---

Table 1: Core features of TeamForge PM

2. Technology Stack

The technology stack for TeamForge PM has been carefully selected to balance developer experience, performance, scalability, and maintainability. Each technology serves a specific purpose in the architecture, and together they form a cohesive ecosystem that supports the full development lifecycle from local development to production deployment. The stack is locked, meaning no new dependencies will be introduced without explicit approval from the project superadmin, ensuring consistency and reducing the risk of dependency conflicts.

2.1 Core Technologies

Technology	Category	Purpose
Next.js 16	Framework	Full-stack React framework with App Router and server components
TypeScript	Language	Strict mode for type safety across the entire codebase
Turso SQLite	Database	Edge-friendly libSQL database with Prisma ORM
Prisma	ORM	Type-safe database access with migrations and schema management
NextAuth.js v5	Auth	JWT-based authentication with credentials provider
Tailwind CSS 4	Styling	Utility-first CSS framework for rapid UI development
shadcn/ui	UI Library	Pre-built accessible React components with Tailwind
Resend	Email	Transactional email API for notifications and alerts
Twilio	SMS	Optional SMS notifications for urgent alerts
z-ai-web-dev-sdk	AI	GLM AI integration for chat and code generation
Vercel	Deployment	Automatic deployments from GitHub with edge hosting

Table 2: Core technology stack

2.2 Supporting Libraries

In addition to the core technologies, the project utilizes several supporting libraries that enhance developer productivity and user experience. Zod provides runtime schema validation for all API inputs and form submissions, ensuring data integrity at every entry point. React Hook Form pairs with Zod to provide a performant form handling solution with minimal re-renders. The date-fns library offers a comprehensive set of date manipulation utilities for formatting timestamps, calculating durations, and displaying relative times in the time tracking module.

For data visualization on the dashboard, Recharts provides a composable charting library built on React components, enabling interactive charts for time tracking analytics, project progress, and team activity metrics. Lucide React supplies a consistent icon set that integrates seamlessly with the design system. Sonner handles toast notifications for user feedback on actions like saving changes, creating projects, or clocking in. For production environments, additional tools like Upstash Redis for rate limiting and Sentry for error tracking are recommended.

3. Database Schema

The database schema is designed around a relational model with nine core tables that capture all aspects of team management, project organization, time tracking, and AI interactions. Prisma serves as the sole data access layer, meaning all database operations must go through Prisma queries rather than raw SQL, ensuring type safety and consistent query patterns across the application. The schema uses cuid for primary key generation, providing globally unique identifiers without requiring database-level auto-increment configuration.

3.1 Entity Overview

Entity	Description
User	Core user accounts with roles, profile info, and active status
Account	NextAuth OAuth account linking (reserved for future OAuth support)
Session	NextAuth session management for JWT-based authentication
Project	Project records with name, description, and status tracking
ProjectMember	Join table linking users to projects with project-specific roles
TimeLog	Time tracking entries with clock-in, clock-out, and duration
ActivityLog	Audit trail recording all user actions with timestamps

Invitation	Email invitation tokens for onboarding new team members
AiChat	Persistent AI conversation history linked to users and projects

Table 3: Database entities

3.2 Role-Based Access Control

The role system defines three levels of access control that determine what each user can see and do within the application. The Superadmin role grants unrestricted access to all system features, including user management, project creation, system settings, and the ability to promote other users to Admin or Superadmin status. This role is intended for the project owner or primary system administrator who needs full visibility and control over the platform.

The Admin role provides elevated privileges focused on project and team management. Admins can create and modify projects, assign team members as leads or regular members, and view time logs for all users within their managed projects. However, Admins cannot modify user accounts, change system settings, or alter role assignments at the global level. The Member role is the most restricted, allowing users to view their assigned projects, clock in and out of work sessions, access their own time logs, and interact with the AI assistant. This ensures that sensitive administrative functions remain protected while still giving team members the tools they need to be productive.

Role	Scope	Capabilities
Superadmin	Full system access	Manage users, projects, settings, roles
Admin	Project management	Create projects, assign members, view time logs
Member	Self-service	Clock in/out, view own projects, use AI assistant

Table 4: Role-based access control hierarchy

4. Project Structure

The project follows a well-organized directory structure that separates concerns across multiple layers of the application. The Next.js App Router structure uses route groups to organize pages by functionality, with a dedicated (auth) group for authentication pages and a (dashboard) group for all protected application pages. This approach enables shared layouts within each group while keeping the URL structure clean and intuitive. API routes follow RESTful conventions, with each resource having its own directory containing a route.ts handler for the standard HTTP methods.

4.1 Directory Layout

The source code lives under the `src/` directory and is divided into four main subdirectories: `app/`, `components/`, `lib/`, and `hooks/`. The `app/` directory contains all Next.js routes including pages, layouts, and API endpoints. Components are organized by feature domain (dashboard, team, projects, time-tracker, ai-assistant) with shared components in the `shared/` subdirectory and auto-generated shadcn/ui components in the `ui/` subdirectory. The `lib/` directory houses configuration files, utility functions, Zod validation schemas, and the Prisma client singleton. Custom React hooks for reusable stateful logic reside in the `hooks/` directory.

Directory	Purpose
<code>src/app/</code>	Next.js App Router pages, layouts, and API routes
<code>src/components/ui/</code>	shadcn/ui auto-generated components
<code>src/components/dashboard/</code>	Dashboard-specific widgets and cards
<code>src/components/team/</code>	Team member management components
<code>src/components/projects/</code>	Project CRUD and assignment components
<code>src/components/time-tracker/</code>	Clock in/out interface components
<code>src/components/ai-assistant/</code>	GLM chat interface components
<code>src/components/shared/</code>	Reusable components (tables, dialogs, loaders)
<code>src/lib/</code>	Database client, auth config, utilities, validations
<code>src/hooks/</code>	Custom React hooks (<code>useCurrentUser</code> , <code>useTimeTracker</code>)
<code>src/types/</code>	Shared TypeScript type definitions
<code>prisma/</code>	Prisma schema, migrations, and seed scripts
<code>docs/</code>	Project documentation (architecture, API, deployment)

Table 5: Key directories and their purposes

5. API Design

The API follows RESTful conventions with a consistent design pattern across all endpoints. Every protected endpoint requires a valid session cookie set by NextAuth, and authorization checks are performed at the route level before any database operations occur. All API responses follow a standardized JSON format with a success boolean, data payload (for successful responses), and an error message (for failures). Input validation is handled exclusively through Zod schemas, which are defined in the `lib/validations/` directory and reused across both client and server code.

5.1 API Endpoints

Method	Endpoint	Description	Auth
POST	/api/auth/[...nextauth]	Authentication (login/register)	Public
GET	/api/users	List all users	Superadmin
POST	/api/users	Create new user	Superadmin
PUT	/api/users/[id]	Update user	Self / Superadmin
GET	/api/projects	List projects	Authenticated
POST	/api/projects	Create project	Admin+
PUT	/api/projects/[id]	Update project	Admin+
DELETE	/api/projects/[id]	Delete project	Superadmin
POST	/api/projects/[id]/members	Assign member to project	Admin+
GET	/api/time-logs	List time logs	Self / Admin+
POST	/api/time-logs	Clock in (start session)	Authenticated
PUT	/api/time-logs/[id]	Clock out (end session)	Authenticated
POST	/api/ai/chat	Send message to GLM	Authenticated
POST	/api/notifications	Send notification	Admin+

Table 6: Complete API endpoint reference

5.2 Response Format

All API endpoints return responses in a consistent JSON format to simplify error handling on the client side. Successful responses include a "success" field set to true along with the requested data payload and an optional message. Error responses set "success" to false and include a human-readable error message

that can be displayed to the user. The API uses standard HTTP status codes to indicate the nature of the response: 200 for successful reads, 201 for resource creation, 400 for validation errors, 401 for missing authentication, 403 for insufficient permissions, 404 for missing resources, 409 for duplicate entries, and 500 for server-side errors.

6. AI Assistant Integration

The AI assistant is powered by GLM and accessed exclusively through server-side API routes using the `z-ai-web-dev-sdk`. This architecture ensures that API keys and SDK configuration remain secure on the server, with only the chat messages being transmitted between the client and the server. The chat interface allows users to ask questions about their projects, request code generation, draft documentation, and get AI-powered suggestions for development tasks.

6.1 Chat Protocol

When a user sends a message, the client POSTs to `/api/ai/chat` with the message content, an optional project ID for context, and the full conversation history for maintaining context. The server saves the user message to the `AiChat` table, constructs a system prompt that includes project context (project name, description, assigned team members, and recent activity), and then invokes the GLM model through the `z-ai-web-dev-sdk`. The assistant response is saved to the `AiChat` table and returned to the client for display in the chat interface.

6.2 Future: Agent Mode

The planned agent mode will extend the AI assistant capabilities significantly. With access to a GitHub Personal Access Token and Vercel API credentials, GLM will be able to read the project repository, create new branches, commit code changes, and trigger deployments directly from the chat interface. This creates an end-to-end development workflow where a team member can describe a feature, have GLM generate and commit the code, and deploy it to production, all from within the TeamForge PM interface. The required environment variables for agent mode are `GITHUB_TOKEN`, `VERCEL_TOKEN`, and `VERCEL_PROJECT_ID`.

7. Time Tracking System

The time tracking system is a core feature of TeamForge PM that enables accurate recording of work hours for every team member. It operates on a simple clock-in/clock-out model where members start a work session by selecting a project and clicking "Clock In," which creates a `TimeLog` record with the

current timestamp as the clockIn value and a null clockOut. When the member finishes work, they click "Clock Out," which sets the clockOut timestamp and automatically calculates the duration in seconds. The system prevents duplicate active sessions by validating that no existing TimeLog with a null clockOut exists for the user.

7.1 Clock In/Out Flow

The clock-in process begins with the member selecting a project from their assigned projects list, optionally adding a note describing the work they plan to do. The POST request to /api/time-logs triggers server-side validation that checks authentication, verifies the user has no active session, and confirms the user is a member of the selected project. If all validations pass, a new TimeLog record is created and an ActivityLog entry is generated to audit the clock-in event. The clock-out process mirrors this flow, finding the active session, updating it with the current timestamp, computing the duration, and logging the event.

7.2 Time Log Analytics

Time log data feeds into the dashboard analytics module, where Recharts renders visualizations of work patterns across the team. Admins and Superadmins can view aggregate time reports filtered by date range, project, or team member. The analytics include total hours worked per day, weekly trends, project distribution of work hours, and comparisons across team members. These insights help management identify workload imbalances, track project progress, and make informed decisions about resource allocation.

8. Notification System

The notification system uses Resend as the primary email delivery service and optionally Twilio for SMS notifications. Notifications are triggered by specific system events and sent to the appropriate recipients based on the event type. The system supports both immediate notifications for time-sensitive events and scheduled notifications for recurring summaries. All notification templates are managed server-side, ensuring consistent branding and formatting across all emails sent from the platform.

Event	Trigger	Recipients	Timing
Welcome Email	New user registered	New user	Immediate
Team Invitation	User invited to project	Invitee	Immediate
Project Assignment	Assigned as lead/member	Assigned user	Immediate

Daily Summary	Every day at 9 AM	All active users	Scheduled (cron)
Inactivity Alert	No clock-in by 10 AM	Superadmin	Scheduled (cron)
Urgent Alert	System down / critical issue	Superadmin	Immediate (SMS)

Table 7: Notification event matrix

9. Security Architecture

Security is a foundational concern in TeamForge PM, and multiple layers of protection are implemented throughout the application stack. Passwords are hashed using bcrypt through the NextAuth credentials adapter, ensuring that plaintext passwords are never stored in the database. Session management uses HTTP-only secure cookies with JWT tokens, preventing session hijacking through XSS attacks. CSRF protection is handled automatically by NextAuth, and all API inputs are validated through Zod schemas before processing, providing defense against injection attacks and malformed data.

The Prisma ORM provides an additional layer of protection against SQL injection through its parameterized query builder, which ensures that user input is never interpolated directly into SQL statements. The middleware layer enforces route protection, ensuring that unauthenticated users cannot access dashboard pages or protected API endpoints. Role-based authorization checks at the API level prevent privilege escalation, where a regular member might attempt to access admin-only functionality by directly calling an API endpoint. For production environments, rate limiting via Upstash Redis is recommended to protect against brute-force attacks on authentication endpoints.

Threat	Mitigation
SQL Injection	Prisma parameterized queries
XSS Attacks	React auto-escaping and CSP headers
CSRF Attacks	NextAuth built-in CSRF tokens
Session Hijacking	HTTP-only secure cookies with JWT
Brute Force	Rate limiting (Upstash Redis in production)
Data Exposure	RBAC on every API route
Password Leakage	bcrypt hashing, no plaintext storage

Table 8: Security threat model and mitigations

10. Deployment Guide

TeamForge PM is designed for deployment on Vercel, which provides seamless integration with GitHub repositories for automatic deployments on every push to the main branch. The deployment process requires configuring several environment variables in the Vercel dashboard, including the Turso database URL and authentication token, NextAuth configuration values, and the Resend API key for email notifications. The Turso database is managed separately from the hosting infrastructure, providing a clear separation between compute and storage layers.

10.1 Environment Variables

Variable	Description	Status
DATABASE_URL	Turso database connection URL	Required
TURSO_DATABASE_URL	Turso production URL	Required
TURSO_AUTH_TOKEN	Turso authentication token	Required
NEXTAUTH_URL	Application base URL	Required
NEXTAUTH_SECRET	JWT signing secret	Required
RESEND_API_KEY	Resend email API key	Required
TWILIO_ACCOUNT_SID	Twilio account ID	Optional
TWILIO_AUTH_TOKEN	Twilio auth token	Optional
GITHUB_TOKEN	GitHub PAT (agent mode)	Future
VERCEL_TOKEN	Vercel API token (agent mode)	Future

Table 9: Required environment variables

10.2 Deployment Checklist

- All environment variables are set in Vercel dashboard
- Database migrations have run successfully with `npx prisma migrate deploy`
- Superadmin seed script has been executed to create initial admin account
- NextAuth secret is a strong random string generated with `openssl rand -base64 32`

- HTTPS is enabled (Vercel provides automatic SSL certificates)
- Email domain is verified in Resend dashboard
- Test all critical flows: register, login, clock in/out, create project, AI chat
- Custom error pages (404, 500) are implemented and tested
- Rate limiting is configured for authentication endpoints

11. Coding Conventions

Consistency in code style is critical for maintainability, especially in a project that may be worked on by multiple developers or AI agents. TeamForge PM enforces a strict set of coding conventions documented in the AGENT.md file, which serves as the authoritative reference for all code contributions. These conventions cover file naming, code style, component patterns, API design, and Git commit message format. Every contribution to the codebase must adhere to these rules without exception.

11.1 Key Rules

Convention	Rule
File Naming	kebab-case for all files (e.g., clock-button.tsx)
TypeScript	Strict mode, no any types, proper interfaces
Exports	Named exports preferred, default only for pages
Variables	const over let, never var
Forms	Zod validation for all inputs, React Hook Form
Components	"use client" when using hooks or browser APIs
API Routes	try/catch all async ops, consistent JSON response
Git Commits	Conventional commits: feat:, fix:, chore:, docs:
Imports	React, third-party, UI components, local utils

Table 10: Key coding conventions

12. Using AGENT.md for Persistent Memory

The AGENT.md file is the most important configuration file in the project. It serves as a persistent memory across all development sessions, ensuring that every AI agent or developer working on the project has immediate access to the complete project context, rules, and conventions without needing to re-explain anything. At the start of every new session, the instruction is simple: "Read AGENT.md first and follow everything in it." This eliminates repetitive briefings and ensures consistency in implementation decisions across sessions.

12.1 What AGENT.md Contains

Section	Content
Project Overview	Name, type, description, primary language, communication style
Tech Stack	Locked technology decisions with versions
Project Structure	Complete directory tree with file descriptions
Database Schema	Full Prisma schema with all tables and relationships
RBAC Rules	Role definitions and permission matrix
Coding Conventions	File naming, code style, API design patterns
AI Protocols	GLM integration rules and chat protocol
Environment Variables	All required and optional configuration
Notification Rules	Email/SMS event triggers and templates
Security Rules	Authentication, authorization, and threat mitigations
Current Status	Phase tracking with completed, in-progress, and pending items
Change Log	History of modifications to AGENT.md itself

Table 11: AGENT.md section overview

12.2 How to Use It

At the beginning of every new development session, tell the AI assistant to read AGENT.md first. The agent will then have complete context about the project, including all technical decisions, conventions, and the current build status. When the project evolves and new decisions are made, update AGENT.md to reflect those changes. This creates a living document that grows with the project. The file also contains a status tracking section that shows which phases are completed, in progress, or pending, giving the agent a clear starting point for each session.

13. Credentials and Setup Requirements

To get TeamForge PM fully operational, several external service credentials are required. These can be provided incrementally as the project progresses through its build phases. The critical credentials needed before production deployment include the Turso database connection details and authentication token, the NextAuth secret for secure session management, and the Resend API key for email notifications. Optional credentials for Twilio SMS, GitHub integration, and Vercel API access can be added later for enhanced functionality and agent mode.

Credential	Format	Priority	Phase
Turso Database URL	libsql://your-db.turso.io	Required	Phase 1
Turso Auth Token	From Turso dashboard	Required	Phase 1
Resend API Key	re_... from Resend	Required	Phase 5
Twilio Credentials	SID + Auth Token	Optional	Phase 9
GitHub Token	ghp_... (PAT with repo)	Future	Agent Mode
Vercel Token	From Vercel dashboard	Future	Agent Mode

Table 12: Required credentials and their priority

If credentials are not available at the start of development, the project can still be built and tested locally using file-based SQLite as the database backend and mock email functionality. The `.env.example` file in the project root provides a template for all required variables with descriptions of where to obtain each value. This allows development to proceed immediately while credentials are gathered and configured at the appropriate stage of the build process.